

2. Fundamento teórico

2.1. Marco normativo

La ingeniería de requisitos se apoya en estándares, cuerpos de conocimiento y marcos metodológicos que establecen principios para especificar, validar, mantener y controlar los requisitos durante el ciclo de vida de un sistema. Entre los principales referentes se encuentran ISO/IEC/IEEE 29148:2018 para los procesos de ingeniería de requisitos, ISO/IEC/IEEE 15288:2023 e ISO/IEC/IEEE 12207:2017 para los procesos del ciclo de vida, ISO/IEC 25010:2023 para las características de calidad, así como SWEBOK v4.0 y los fundamentos contemporáneos de ingeniería de requisitos desarrollados por Pohl. Estos referentes permiten abordar la calidad de los requisitos no únicamente desde su redacción, sino también desde su verificabilidad, trazabilidad y evolución durante el desarrollo.

La relevancia de estos mecanismos continúa siendo respaldada por investigaciones recientes. Un estudio de mapeo publicado en *Requirements Engineering*, señalan que la calidad de los requisitos constituye un área multidimensional cuya evaluación involucra diferentes propiedades, defectos y mecanismos de aseguramiento [1]. Esta perspectiva justifica la utilización conjunta de estándares y prácticas de ingeniería para establecer criterios consistentes de calidad y control de los requisitos.

2.1.1. ¿Qué regula la validación y gestión de requisitos?

La validación y la gestión de requisitos comprenden actividades destinadas a asegurar que los requisitos representen adecuadamente las necesidades de los interesados y permanezcan controlados durante la evolución del proyecto. La validación se orienta a determinar si los requisitos especificados corresponden con las necesidades y objetivos que el sistema debe satisfacer, mientras que la gestión considera aspectos como los atributos de los requisitos, su trazabilidad, priorización, versionado y tratamiento de cambios.

Estas actividades no deben entenderse como procesos aislados. La modificación de un requisito puede afectar otros requisitos y artefactos asociados, por lo que resulta necesario mantener relaciones que permitan conocer su procedencia y las consecuencias de su evolución. Investigaciones recientes sobre ingeniería de requisitos identifican precisamente la validación y la gestión como actividades fundamentales: la primera busca comprobar la calidad y adecuación de la información obtenida, mientras que la segunda permite seguir los cambios producidos en los requisitos y mantenerlos acordes con las necesidades de los stakeholders [2].

2.1.2. IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software

ISO/IEC/IEEE 29148:2018 constituye uno de los principales referentes normativos para la ingeniería de requisitos de sistemas y software. El estándar establece procesos y disposiciones relacionadas con la identificación, formulación, análisis y gestión de requisitos a lo largo del ciclo de vida [3]. Dentro de este contexto, la validación permite

determinar si los requisitos y la especificación resultante representan adecuadamente las necesidades de los interesados y los objetivos establecidos para el sistema.

La calidad de un requisito implica que pueda comprenderse y utilizarse posteriormente para actividades de diseño, implementación y comprobación. Por esta razón, la especificación debe procurar características que reduzcan ambigüedades, inconsistencias y dificultades de interpretación. La investigación empírica reciente confirma que la calidad de requisitos continúa siendo un problema relevante en ingeniería de software y que sus atributos y defectos requieren métodos sistemáticos de evaluación [1].

2.1.3. IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios

ISO/IEC/IEEE 15288:2023 proporciona un marco de procesos para el ciclo de vida de sistemas, mientras que ISO/IEC/IEEE 12207:2017 establece procesos aplicables al ciclo de vida del software [4]. Dentro de este marco, los requisitos no se consideran elementos estáticos: pueden evolucionar como consecuencia de nuevas necesidades, restricciones, decisiones técnicas o cambios en el contexto del sistema.

Por ello, el control de cambios debe permitir identificar una modificación, analizar sus posibles consecuencias y mantener consistentes los elementos afectados. En ingeniería de requisitos esto adquiere especial importancia porque una modificación puede repercutir en requisitos relacionados, diseño, implementación y pruebas. Consecuentemente, la trazabilidad constituye un mecanismo de apoyo fundamental para determinar dichas relaciones.

Esta necesidad está respaldada por estudios contemporáneos sobre trazabilidad. Señalan que mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme aumenta el tamaño y complejidad del proyecto. La trazabilidad permite recuperar información sobre el origen, las modificaciones y las personas involucradas en un requisito, proporcionando información necesaria para gestionar su evolución [5].

2.1.4. ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS

ISO/IEC 25010:2023 establece un modelo de calidad que permite caracterizar y evaluar la calidad de productos software y sistemas [6]. Aunque el estándar no sustituye a ISO/IEC/IEEE 29148 como marco específico de ingeniería de requisitos, sus características de calidad pueden utilizarse para orientar la definición de requisitos no funcionales y atributos de calidad dentro de una Especificación de Requisitos de Software [3].

Esto significa que características de calidad relevantes para el producto deben transformarse en requisitos suficientemente claros y evaluables. De esta forma, la ERS actúa como vínculo entre las necesidades de calidad identificadas y las posteriores actividades de diseño, implementación y pruebas.

La importancia de definir correctamente estos atributos está respaldada por la investigación reciente sobre calidad de requisitos. En otro estudio encontraron una diversidad considerable de atributos y problemas utilizados para estudiar empíricamente

la calidad de los requisitos, lo que evidencia la necesidad de utilizar marcos estructurados para definir y evaluar sus propiedades [1].

2.1.5. SWEBOK v4.0 y Pohl (2025) Parte V

El *Guide to the Software Engineering Body of Knowledge* (SWEBOK) v4.0, publicado por IEEE Computer Society en 2024, consolida el conocimiento generalmente aceptado dentro de la ingeniería de software. Su área dedicada a requisitos contempla fundamentos, elicitación, análisis y otras actividades necesarias para transformar necesidades de los interesados en requisitos que puedan ser utilizados durante el desarrollo del software [7], [8]. La versión 4.0 actualiza además el cuerpo de conocimiento para incorporar prácticas contemporáneas de ingeniería de software.

Complementariamente, se presenta en la segunda edición de *Requirements Engineering: Fundamentals, Principles, and Techniques*, publicada por Springer en 2025, un tratamiento sistemático de la ingeniería de requisitos. Particularmente, la Parte V, denominada *Cross-Sectional Activities*, desarrolla cuatro actividades de validación y tres actividades relacionadas con la gestión de requisitos. Este planteamiento permite comprender ambas actividades como elementos transversales que acompañan la evolución de los requisitos y no únicamente como tareas realizadas al finalizar su especificación [9].

La combinación de SWEBOK v4.0 y Pohl proporciona, por tanto, una perspectiva complementaria al marco normativo: mientras SWEBOK sintetiza el cuerpo de conocimiento aceptado de la disciplina, Pohl profundiza en principios, procedimientos y técnicas específicas para validar y gestionar requisitos.

2.1.6. El CCB según PMBOK 7.ª edición

El control formal de cambios requiere establecer responsabilidades para evaluar las modificaciones propuestas antes de incorporarlas a una línea base. En este contexto se utiliza el Change Control Board (CCB) o Comité de Control de Cambios, entendido como un grupo formalmente establecido al que se asigna autoridad para evaluar determinadas solicitudes de cambio conforme al procedimiento definido para el proyecto.

Aplicado a la ingeniería de requisitos, una solicitud de modificación debe registrarse y someterse a análisis antes de modificar directamente la ERS o su línea base. El análisis de impacto permite determinar qué requisitos, artefactos, componentes, costos, plazos o pruebas podrían verse afectados. Con esa información puede adoptarse una decisión sobre la aprobación, rechazo o aplazamiento del cambio y, cuando corresponda, actualizar los elementos afectados manteniendo su trazabilidad.

2.2. Validación de requisitos

La validación de requisitos constituye una actividad esencial de la ingeniería de requisitos orientada a determinar si la especificación representa adecuadamente las necesidades, expectativas y objetivos de los interesados. Su importancia radica en detectar problemas antes de que estos se propaguen hacia etapas posteriores del desarrollo, donde su

corrección puede implicar modificaciones en el diseño, implementación y pruebas del sistema.

La literatura reciente continúa considerando la validación como una de las actividades fundamentales de la ingeniería de requisitos. Señalan que la validación busca garantizar que la información recopilada se encuentre correctamente organizada para satisfacer los objetivos del sistema, evaluando aspectos como exactitud, completitud y corrección de los documentos o modelos de requisitos [10].

Esta actividad se relaciona directamente con la calidad del ERS. Identifican entre los atributos estudiados con mayor frecuencia en la calidad de requisitos la ambigüedad, completitud, consistencia y corrección, características cuya evaluación permite descubrir deficiencias antes de utilizar la especificación como base para las siguientes etapas del desarrollo [1].

2.2.1. Verificación vs. validación: diferencias conceptuales

Aunque los términos verificación y validación suelen aparecer conjuntamente, representan objetivos diferentes dentro del aseguramiento de la calidad. La verificación se concentra principalmente en comprobar si los requisitos han sido especificados de manera adecuada, es decir, si presentan las propiedades necesarias para poder ser comprendidos, analizados y posteriormente comprobados. En este proceso pueden examinarse características como consistencia, completitud, ausencia de ambigüedad y verificabilidad.

La validación, en cambio, se orienta hacia la correspondencia entre los requisitos documentados y las necesidades reales de los stakeholders. Por tanto, un requisito puede estar correctamente redactado desde el punto de vista formal y aun así no representar lo que necesita el usuario. La validación requiere entonces involucrar a los interesados para determinar si lo especificado describe el sistema que realmente se pretende desarrollar.

La diferencia puede sintetizarse en dos preguntas: la verificación analiza si los requisitos están correctamente especificados, mientras que la validación determina si se han especificado los requisitos adecuados. Sin embargo, ambas actividades son complementarias y contribuyen conjuntamente al aseguramiento de la calidad del ERS.

Muestran la importancia de la verificabilidad mediante un enfoque destinado específicamente a verificar requisitos de rendimiento y generar entornos de prueba a partir de ellos, evidenciando la relación existente entre requisitos correctamente formulados y su posterior comprobación [11].

2.2.2. Principios de validación de requisitos

La validación debe realizarse de manera sistemática y considerar tanto las características individuales de cada requisito como la consistencia del conjunto completo. Un requisito aislado puede parecer adecuado, pero generar contradicciones cuando se relaciona con otros elementos del ERS. Por ello, la revisión debe considerar la especificación desde una perspectiva integral.

Entre los aspectos fundamentales se encuentran la corrección, para determinar si el requisito representa una necesidad real; la completitud, para comprobar que la información necesaria haya sido especificada; la consistencia, para identificar contradicciones entre requisitos; y la ausencia de ambigüedad, de modo que un requisito no admita interpretaciones incompatibles.

Estos aspectos están respaldados por el estudio sistemático identificaron doce atributos de calidad estudiados empíricamente y encontraron que ambigüedad, completitud, consistencia y corrección se encuentran entre los más prominentes en la investigación sobre calidad de requisitos [1].

La validación tampoco debe concebirse exclusivamente como una actividad final. Los requisitos evolucionan y pueden ser modificados como resultado de nuevas necesidades o de la detección de inconsistencias. Por ello, resulta conveniente realizar actividades de validación de manera iterativa durante su elaboración y después de cambios relevantes.

2.2.3. Técnicas de validación

La validación de requisitos puede realizarse mediante diferentes técnicas cuya selección depende de las características del proyecto, los artefactos disponibles y la participación de los stakeholders. Entre las alternativas se encuentran las revisiones e inspecciones, el uso de prototipos, la construcción y evaluación de escenarios, la utilización de listas de comprobación y la derivación temprana de casos o criterios de prueba.

Las revisiones permiten que diferentes participantes examinen los requisitos para localizar omisiones, contradicciones, ambigüedades o información incorrecta. Los prototipos permiten representar anticipadamente determinadas características o interacciones del sistema, facilitando que los usuarios evalúen si el comportamiento esperado corresponde con sus necesidades. Los escenarios, por su parte, describen situaciones concretas de utilización y permiten analizar cómo los requisitos se manifiestan durante una interacción determinada.

La investigación reciente demuestra que las prácticas de validación continúan evolucionando según el contexto tecnológico. En un estudio de mapeo sobre ingeniería de requisitos para sistemas basados en inteligencia artificial, identificaron 53 estudios relacionados con validación de requisitos, encontrando prácticas como entrevistas, cuestionarios y escenarios, además de nuevas propuestas adaptadas a estos sistemas [12].

La utilización conjunta de diferentes técnicas puede resultar especialmente útil, debido a que una sola técnica difícilmente permite detectar todos los tipos de defectos presentes en una especificación.

2.2.4. Inspección Fagan: origen, fases y roles

La Inspección Fagan constituye un método formal y estructurado de revisión introducido originalmente por Michael Fagan para la detección sistemática de defectos en artefactos de software. Aunque su origen es anterior al intervalo bibliográfico establecido para los artículos de este trabajo, continúa siendo utilizado como referente conceptual en investigaciones contemporáneas relacionadas con inspecciones.

Aplicada a un ERS, la inspección permite examinar sistemáticamente los requisitos para identificar defectos antes de avanzar hacia etapas posteriores. El proceso puede organizarse en actividades de planificación, preparación individual, reunión de inspección, corrección o retrabajo y seguimiento. Durante estas actividades se distribuyen responsabilidades entre participantes como el autor del artefacto, el moderador o responsable de conducir la inspección, los inspectores o revisores y, cuando se establece como función independiente, el encargado de registrar los defectos encontrados.

La finalidad principal de la inspección no consiste en rediseñar inmediatamente el requisito durante la reunión, sino en identificar y registrar sus defectos para que posteriormente puedan ser corregidos y verificados.

La vigencia de la inspección de requisitos puede observarse en investigaciones recientes. Estudiaron los fallos dependientes producidos durante la inspección de requisitos y destacan que la especificación suele ser examinada por un equipo de inspectores para detectar defectos, debido a la influencia que su calidad ejerce sobre las fases posteriores y sobre el producto software [13].

2.2.5. Métricas de inspección

Las métricas de inspección permiten evaluar cuantitativamente los resultados de una revisión y proporcionan información para valorar la efectividad del proceso. Entre las medidas que pueden registrarse se encuentran el número total de defectos identificados, su clasificación por tipo o severidad, el tiempo empleado en la inspección, la cantidad de requisitos examinados y la distribución de defectos entre las diferentes partes del ERS.

A partir de estas medidas pueden construirse indicadores como la densidad de defectos, relacionando la cantidad de defectos encontrados con el tamaño del artefacto inspeccionado. También puede analizarse la productividad de la revisión considerando los defectos identificados respecto al esfuerzo invertido.

No obstante, una cantidad elevada de defectos detectados no necesariamente significa que el proceso de requisitos sea de mejor calidad; puede indicar que la inspección ha sido efectiva, pero también que el documento inicial contenía numerosos problemas. Por ello, las métricas deben interpretarse conjuntamente y compararse entre revisiones o versiones del ERS.

Además, la necesidad de considerar la composición del equipo de inspección, puesto que la incorporación de varios inspectores no garantiza que sus detecciones sean completamente independientes entre sí [13].

2.2.6. Caso de estudio o ejemplo aplicado

Como ejemplo de aplicación, considérese la validación de un requisito perteneciente al proyecto MundiPets:

RF-01: El sistema permitirá registrar una mascota asociándola con los datos correspondientes a su propietario.

Durante una revisión del requisito, el equipo podría identificar que la expresión “datos correspondientes” no especifica cuáles son obligatorios ni establece qué debe ocurrir si el propietario todavía no se encuentra registrado. Además, tampoco indica restricciones sobre la información de la mascota ni las condiciones que determinan que el registro se haya realizado correctamente.

Como resultado de la validación, el requisito podría descomponerse o complementarse con criterios específicos relacionados con los datos obligatorios, las validaciones de entrada y el comportamiento esperado ante situaciones excepcionales. Posteriormente, los stakeholders deberían confirmar que las modificaciones representan correctamente el proceso que desean implementar.

Este ejemplo evidencia que la validación no se limita a comprobar la gramática del requisito. Su objetivo es descubrir información ausente, ambigua, inconsistente o incorrecta antes de que el requisito sea utilizado como fundamento para el diseño y desarrollo. Además, cualquier modificación resultante deberá incorporarse mediante los mecanismos de gestión de requisitos tratados en **2.3**, manteniendo su versión y trazabilidad.

2.3. Gestión de requisitos

La gestión de requisitos comprende el conjunto de actividades destinadas a mantener los requisitos identificados, organizados, actualizados y trazables durante el ciclo de vida del sistema. Su importancia aumenta conforme evoluciona el proyecto, debido a que los requisitos pueden modificarse como consecuencia de nuevas necesidades de los stakeholders, restricciones técnicas, cambios del entorno o decisiones tomadas durante el desarrollo.

Gestionar los requisitos implica, por tanto, mantener información sobre su estado, origen, prioridad, versión y relaciones con otros elementos del proyecto. La trazabilidad constituye uno de los principales mecanismos para conseguirlo, ya que permite establecer vínculos entre los requisitos y los artefactos que los originan o que posteriormente se producen a partir de ellos.

Mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme los proyectos aumentan de tamaño y complejidad. La trazabilidad permite responder preguntas relacionadas con el origen de un requisito, los motivos de sus modificaciones y las personas involucradas en su definición [5].

2.3.1. Gestión de la configuración del ERS

La Especificación de Requisitos de Software (ERS) constituye uno de los principales artefactos documentales de la ingeniería de requisitos. Debido a que puede evolucionar durante el proyecto, resulta necesario gestionar su configuración para identificar las versiones existentes y mantener control sobre las modificaciones realizadas.

La gestión de configuración del ERS permite determinar qué versión se encuentra vigente, qué requisitos fueron modificados, cuándo se produjeron los cambios y cuáles son los elementos afectados. De esta manera, se evita que diferentes miembros del equipo

trabajen con versiones incompatibles de la especificación y se conserva un historial de su evolución.

Este control adquiere especial importancia cuando los requisitos mantienen relaciones con otros artefactos. Un cambio realizado en la ERS puede repercutir posteriormente en elementos de diseño, código o pruebas, por lo que la configuración y la trazabilidad deben mantenerse coordinadas.

2.3.2. Línea base (baseline) y control de versiones

Una línea base o baseline representa una versión formalmente establecida de un conjunto de requisitos que sirve como referencia para continuar el desarrollo. Una vez establecida, sus elementos no deberían modificarse de manera informal; los cambios posteriores deben someterse al procedimiento de control definido por el proyecto.

La línea base no significa que los requisitos sean inmutables. Su propósito consiste en proporcionar un punto de referencia conocido frente al cual puedan identificarse y evaluar las modificaciones posteriores. Así, cuando se aprueba un cambio, se registra la modificación y se genera una nueva versión manteniendo el historial correspondiente.

El control de versiones complementa este mecanismo al permitir identificar la evolución del ERS y de sus requisitos. Como mínimo, debe ser posible conocer qué elemento cambió, cuál era su contenido anterior, cuál es su estado actual y qué modificación dio lugar a la nueva versión.

2.3.3. Atributos de los requisitos

Además de su descripción textual, los requisitos pueden incorporar atributos que proporcionan información adicional para facilitar su gestión. Estos atributos permiten identificar, clasificar, priorizar y controlar los requisitos durante su evolución.

Entre los atributos que pueden asociarse a un requisito se encuentran un identificador único, estado, prioridad, responsable, fuente u origen, versión, fecha de creación o modificación y criterios asociados a su verificación. La selección concreta de atributos dependerá de las necesidades del proyecto y de su proceso de gestión.

Por ejemplo, un requisito funcional podría registrarse de la siguiente manera:

Identificador: RF-01

Descripción: Registrar mascota.

Prioridad: Alta.

Fuente: Stakeholder.

Estado: Aprobado.

Versión: 1.1.

Esta información facilita posteriormente operaciones como localizar requisitos, identificar modificaciones, determinar responsabilidades y establecer relaciones de trazabilidad. Por ello, los atributos no sustituyen al contenido del requisito, sino que proporcionan los metadatos necesarios para administrarlo durante su ciclo de vida.

2.3.4. Trazabilidad: pre-traceability y post-traceability

La trazabilidad de requisitos es la capacidad de establecer y mantener relaciones entre un requisito y la información vinculada con su origen, evolución e implementación. Estas relaciones permiten comprender por qué existe determinado requisito y qué elementos podrían verse afectados si se modifica.

Entre pre-requirements specification traceability (pre-RS) y post-requirements specification traceability (post-RS). La primera establece relaciones entre los requisitos y sus fuentes anteriores a su incorporación en la especificación, como entrevistas con stakeholders, reuniones, documentos organizacionales o sistemas existentes [5].

Por ejemplo:

Entrevista con propietario → necesidad identificada → RF-01 Registrar mascota.

La post-traceability, por otra parte, establece relaciones entre los requisitos especificados y los artefactos generados posteriormente durante el desarrollo, como elementos de diseño, código fuente y casos de prueba.

Por ejemplo:

RF-01 Registrar mascota → módulo de registro → código correspondiente → CP-01 Prueba de registro.

Por tanto, ambas perspectivas permiten reconstruir el ciclo del requisito: la pre-traceability ayuda a comprender de dónde proviene, mientras que la post-traceability permite conocer cómo fue desarrollado y comprobado.

La investigación reciente continúa demostrando la importancia y, al mismo tiempo, las dificultades prácticas de mantener estos vínculos. Las barreras que dificultan la adopción de trazabilidad en proyectos reales, mientras que otro estudio identificaron específicamente una menor atención histórica hacia la pre-trazabilidad en comparación con la trazabilidad posterior a la especificación [5], [14].

2.3.5. Matriz de trazabilidad

Una matriz de trazabilidad de requisitos constituye una representación estructurada de las relaciones existentes entre requisitos y otros elementos del proyecto. Su objetivo es facilitar la comprobación de cobertura y permitir identificar rápidamente qué artefactos están relacionados con determinado requisito.

Una representación simplificada para MundiPets podría ser:

ID	Requisito	Fuente	Diseño/Módulo	Caso de prueba	Estado
RF-01	Registrar mascota	Entrevista E-01	Gestión de mascotas	CP-01	Validado

RF-02	Consultar mascota	Entrevista E-02	Gestión de mascotas	CP-02	Validado
RF-03	Actualizar datos	Entrevista E-02	Gestión de mascotas	CP-03	Pendiente

La matriz permite realizar tanto seguimiento hacia adelante como hacia atrás. Por ejemplo, partiendo de RF-01 puede localizarse el módulo y caso de prueba relacionados; en sentido contrario, partiendo de CP-01 puede identificarse qué requisito justifica la existencia de dicha prueba.

Esta información también resulta útil durante el análisis de cambios. Si RF-01 se modifica, los enlaces permiten identificar los artefactos potencialmente afectados y determinar cuáles deben revisarse.

No obstante, mantener trazabilidad tiene un costo organizacional. encontraron mediante un estudio empírico que, aunque existen numerosas técnicas y herramientas de trazabilidad, su utilización práctica continúa enfrentándose a distintas barreras [14].

2.3.6. Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre

Los requisitos pueden cambiar durante el desarrollo, pero sus modificaciones deben gestionarse de manera controlada cuando forman parte de una línea base. Un procedimiento estructurado evita que una modificación sea incorporada sin conocer previamente sus consecuencias.

El proceso puede comenzar mediante una Request for Change (RFC) o solicitud de cambio, en la cual se registra la modificación propuesta y su justificación. Posteriormente se realiza un análisis de impacto, considerando los requisitos relacionados y los artefactos que podrían verse afectados, además de factores como esfuerzo, costo, riesgos y planificación.

Con esta información, el Change Control Board (CCB) o la autoridad establecida por el proyecto puede evaluar la solicitud y decidir su aprobación, rechazo o aplazamiento. Si el cambio es aprobado, se procede a su implementación, actualizando el requisito y los elementos relacionados. Finalmente se realiza el cierre, verificando que las modificaciones hayan sido efectuadas correctamente, actualizando las versiones y conservando la trazabilidad del cambio.

De manera resumida, el flujo puede representarse como:

RFC → registro → análisis de impacto → evaluación del CCB → aprobación/rechazo → implementación → verificación → actualización de línea base → cierre.

La trazabilidad desempeña un papel fundamental durante este procedimiento porque permite determinar las posibles consecuencias de modificar un requisito. La literatura reciente señala precisamente que los enlaces de trazabilidad ayudan a comprender el

contexto y evolución de los requisitos, incluyendo las razones que motivaron determinadas modificaciones.

2.3.7. Métricas de volatilidad de requisitos

La volatilidad de requisitos representa el grado en que los requisitos cambian durante un período determinado del proyecto. Su seguimiento permite identificar situaciones en las que se están produciendo modificaciones frecuentes y analizar si estas pueden afectar la estabilidad del desarrollo.

Entre los elementos que pueden contabilizarse se encuentran los requisitos añadidos, modificados y eliminados durante un período. A partir de estos datos pueden establecerse indicadores relativos respecto al número total de requisitos existentes en la línea base.

Por ejemplo, para fines de seguimiento interno podría utilizarse una razón como:

$$V = Ra + Rm + Re / Rt * 100$$

Donde Ra representa los requisitos añadidos, Rm los modificados, Re los eliminados y el Rt número de requisitos tomado como referencia para el período. Esta expresión debe entenderse como un indicador operativo de seguimiento y no como una fórmula universal establecida por ISO/IEC/IEEE 29148.

Si una línea base contiene 50 requisitos y durante un período se añaden 2, se modifican 3 y se elimina 1, el indicador sería:

$$V = 2+3+1 / 50 * 100 = 12\%$$

El resultado indica que durante ese período se produjo actividad de cambio equivalente al 12 % del conjunto tomado como referencia. Sin embargo, el porcentaje no debe interpretarse aisladamente como bueno o malo; debe analizarse junto con la etapa del proyecto, las causas de los cambios y sus impactos.

Estudiaron específicamente la Requirement Change Volatility (RCV) y señalan que una planificación insuficiente frente a cambios destinados a responder a las expectativas de los stakeholders puede producir propagación de modificaciones y contribuir a problemas en el proyecto. Su investigación utiliza redes de requisitos y técnicas de aprendizaje automático para evaluar y predecir esta volatilidad [15].

De esta manera, las métricas de volatilidad complementan la gestión de cambios: mientras la RFC y el análisis de impacto permiten administrar una modificación concreta, las métricas permiten observar el comportamiento agregado de los cambios a lo largo del proyecto.

Bibliografía

- [1] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz, and W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requir. Eng.*, vol. 27, no. 2, pp. 183–209, Jun. 2022, doi: 10.1007/s00766-021-00367-z.
- [2] M. A. Umar and K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requir. Eng.*, vol. 29, no. 2, pp. 177–207, Jun. 2024, doi: 10.1007/s00766-023-00411-0.
- [3] ISO/IEC/IEEE, “29148:2018 — Requirements engineering,” Jul. 2018, *IEEE*. doi: 10.1109/IEEESTD.2018.8559686.
- [4] ISO/IEC/IEEE, “15288:2023 — System life cycle processes,” 2023.
- [5] J. Mucha, A. Kaufmann, and D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requir. Eng.*, vol. 29, no. 2, pp. 119–141, Jun. 2024, doi: 10.1007/s00766-023-00412-z.
- [6] ISO/IEC, “25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023.
- [7] H. Washizaki and J. Olszewska, “Guide to the Software Engineering Body of Knowledge v4.0,” vol. 4.0, p. 413, 2024.
- [8] H. Washizaki, *SWEBOK Guide v4.0*. IEEE Computer Society, 2024. [Online]. Available: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [9] Klaus. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025.
- [10] M. A. Umar and K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requir. Eng.*, vol. 29, no. 2, pp. 177–207, Jun. 2024, doi: 10.1007/s00766-023-00411-0.
- [11] W. Abdeen, X. Chen, and M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *Requir. Eng.*, Apr. 2022, doi: 10.1007/s00766-022-00379-3.
- [12] U.-Habiba, M. Haug, J. Bogner, and S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requir. Eng.*, vol. 29, no. 4, pp. 567–600, Dec. 2024, doi: 10.1007/s00766-024-00432-3.

- [13] B. Li and C. Smidts, "A Zone-Based Model for Analysis of Dependent Failures in Requirements Inspection," *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 3581–3598, Jun. 2023, doi: 10.1109/TSE.2023.3266157.
- [14] M. Ruiz, J. Y. Hu, and F. Dalpiaz, "Why don't we trace? A study on the barriers to software traceability in practice," *Requir. Eng.*, vol. 28, no. 4, pp. 619–637, Dec. 2023, doi: 10.1007/s00766-023-00408-9.
- [15] P. H. Hein, E. Kames, C. Chen, and B. Morkos, "Employing machine learning techniques to assess requirement change volatility," *Res. Eng. Des.*, vol. 32, no. 2, pp. 245–269, Apr. 2021, doi: 10.1007/s00163-020-00353-6.